Code optimization

Machine Independent	Machine Dependent
(1) Loop optimization (a) Code motion (or) Frequency reduction (b) Loop unrolling. (c) Loop Jamming (2) Folding. (3) Redundancy elimination (4) Strength Reduction (5) Algebric simplification	(a) Register Allocation (2) Use of Addressing modes (3) peephole optimization (a) Redundant LOAD/STORE (b) Flow of control optizations (c) use of machine idioms

(1) Loop optimization:-

(1) To apply loop optimization we must first detect loops.

(2) For detecting loops we use control flow analysis using program flow graph.

(3) To find program flow graph, we need to find Basic Blocks.

(4) A Basic Block is a sequence of 3-address statements where control enters at the begining and leaves only at the end without any jumps (or) halts

Basic Blocks-

→ In order to find basic blocks, we need to find leaders in the three address code.

→ A basic block will start from one leader to the next leader but not including next leader.

Finding Leaders in a Basic Block:-

(1) The first three address instruction in the intermediate code is a leader.

(2) Any instruction that is the target of conditional (or) unconditional jump is a leader.

(3) Any instruction that immediately follows a conditional (or) unconditional jump is a leader.

Eg:-

```
fact(n)
{ int f=1
for(i=2 i<=n; i++)
{ f=f*i;
} return f;
```

Three address code:-

1.) f=1
2.) i=2
3.) if(i>n) goto 8
4.) t1=f*i
5.) f=t1
6.) i=i+1
7.) goto 3
8.) goto calling fn.

*1. f=1

2. i=2

B_1

*3. if(i>n) goto 8 B_2

*4. $t_1 = f * i$

5. $f = t_1$

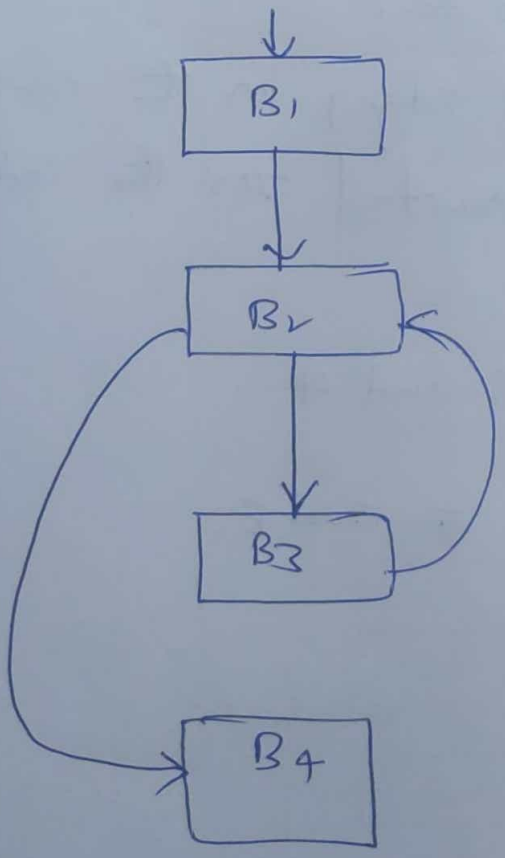
6. $i = i+1$

*7. goto 3

B_3

*8. goto calling fn B_4

Program flow graph:



Gate-2015- Consider the intermediate code given below.

1. i=1
2. j=1
3. $t_1 = 5 * i$
4. $t_2 = t_1 + j$
5. $t_3 = 4 * t_2$
6. $t_4 = t_3$
7. $a[t_4] = -1$
8. j=j+1
9. if (j<=5) goto 3
10. i=i+1
11. if (i<5) goto 2.

The number of nodes and edges in the control flow graph ~~Conditional~~ constructed for the code respectively are.

a) 5 and 7 (b) 6 and 7

(3) 5 and 5 (4) 7 and 8.

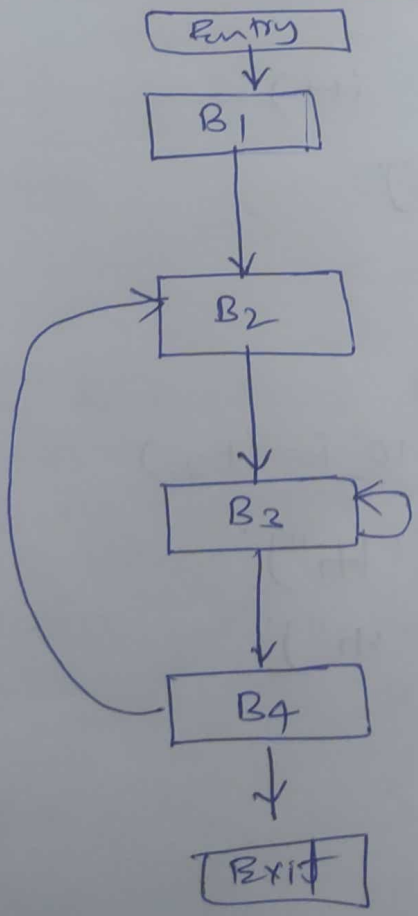
Sol

```
*1.   i = 1                    B1

*2.   j = 1                    B2

*3.   t1 = 5 * i
 4.   t2 = t1 + j
 5.   t3 = 4 * t2              B3
 6.   t4 = t3
 7.   a[t4] = -1
 8.   j = j + 1
 9.   if (j <= 5) goto 3

*10.   i = i + 1
 11.   if (i < 5) goto 2       B4
```



(a) Frequency Reduction (Code motion):-

A statement (or) expression which can be moved outside the loop body without affecting the semantic of the program.

Ex:-

```
i=0                          i=0;
while(i<1000)                t= a/b
{                      ⇒     while(i<1000)
  A=(a/b)+i                  {
  i++                          A= t+i;
}                              i++;
                             }
```

(b) Loop unrolling:- Reducing the no of times comparisons are made in the loop.

Eg:-

```
for(i=0; i<10; i++)
{  printf("\ni");
}
```

⇓

```
for(i=0; i<10; i=i+2)
{   printf("\ni");
    printf("\ni");
}
```

⑤ Loop Jamming:- Combine the bodies of two loops.

Ex:-

```
for(i=0; i<5; i++)
{ a=i+5
}

for(i=0; i<5; i++)
{ b=i+10;
}
```

⇒

```
for(i=0; i<5; i++)
{ a=i+5;
  b=i+10;
}
```

② Constant Folding:- Replacing an expression that can be computed at ~~comptie~~ compile time by its value.

Ex:- `c=2*3.14*r` ⇒ `c=6.28*r;`

`2+3+B+C` ⇒ `5+B+C`

③ Redundancy elimination:-

```
a=b+c          a=b+c
e=d+b+c   ⇒    e=d+a
```

④ strength Reduction:- Replacing a expensive operator by cheaper one.

Expensive	cheaper
x^2 ⇒	x*x
2*a ⇒	a+a
a/2 ⇒	a*0.5
B=A*2 ⇒	B=A<<1

(5) Algebric Simplification:

$x+0 = 0+x = x$, $x-0 = x$

$x*1 = 1*x = x$, $x/1 = x$

Peephole optimization: perform on a small set of instruction (peephole (or) window)

(a) Redundant LOAD / STORE elimination:

(i) $a = b+c$

```
Load R0, b
Add R0, c
store a, R0
```

(ii) $a = b+c$

$d = a+e$

```
Load R0, b
Add R0, c
store a, R0
Load R0, a   ×
Add R0, e
Store d, R0
```

(6) Flow and Control optimization

(1) Avoid Jumps on Jumps

```
L1: Jump L4
  ≡
L2: Jump L3
  ≡
L3: Jump L4
  ≡
L4: x = a+b*c
```

(2) Eliminate Dead code

```
int i=0;
if (i==1)
{ printf("Deadcode");
}
```